

SWORD2 filtering

Tist [SWORD2 interpretacija rezultatov](#) je postal clustered zato bom tu nadaljeval. Izbiranje particij:

- 2 ali več domen
- razmerje približno 1:2 med domenami (ne smejo biti prevelike/premajhne)
- dobre AUL vrednosti domen

JSON parsing

Naredil sem python script `build_decompositions_csv.py` , da mi ustvari CSV z vsemi potrebnimi podatki:

- ime proteina
- ambiguity index
- št. particije
- št. domene
- AUL domene
- start domene (indeks aminokislino)
- konec domene

Struktura podatkov glede na primer 16pk_A:

```
import csv
import glob
import json

# zgradil bo CSV iz JSON datotek. vrstice se grupirajo po proteinih in
# particijah optimalna particija ima vrednost 0. Domain predstavlja
# število zaporedne domene. 1 = prva domena. range start-end domene je glede
# na prvo aminokislino prve PU in zadnjo aminokislino zadnje PU. AUL je od
# domene. A-index in Quality sta 0, če je prazen string, drugače je število
# zvezdic '*'.
#
# primer za 1a62_A:
#
#   protein aindex partition quality domain AUL start end
#   1a62_A  1      0          0        1     81  1   130<---| optimal
#   1a62_A  1      1          1        1     70  1   47<----| alt. 1
#   1a62_A  1      1          1        2      0  48   94      |
#   1a62_A  1      1          1        3     46  95  130      |
#   1a62_A  1      2          3        1     72  1   47<----| alt. 2
#   1a62_A  1      2          3        2      8  48  130      |
#   1a62_A  1      3          1        1     76  1  130<---| alt. 3
#   1a62_A  1      3          1        2      0  48   94      |

field_names = [
    "protein",
    "aindex",
    "partition",
    "quality",
    "domain",
    "AUL",
    "start",
    "end",
]
csv_name = "sword_results.csv"

def domain_bounds(domain) -> tuple[int, int]:
    begin = 0
    end = 0

    for i, pu in enumerate(domain["PUs"]):
        pu_s, pu_e = pu.split("-")
        pu_s, pu_e = int(pu_s), int(pu_e)
        # če je prva, vzami start index
        if i == 0:
            begin = pu_s
        # najdi zadnjo aminokislino
```

```

        if pu_e > end:
            end = pu_e

    return begin, end

def process_partition(part) → list:
    quality = len(part["Quality"])
    domains = part["Domains"]
    dom_list = []

    for i, domain in enumerate(domains.values()):
        aul = domain["AUL"]
        begin, end = domain_bounds(domain)
        dom_list.append([quality, i + 1, aul, begin, end])

    return dom_list

def process_report(fname: str) → list:
    # globalni podatki
    pname = fname.removeprefix("sword_output/").removesuffix("/SWORD2_summary.json")[:6]
    fp = open(fname, "r")
    obj = json.load(fp)
    aidx = obj["Ambiguity index"]

    # pojdi čez vsako particijo in izvleci podatke
    part_reports = []
    pid = 0
    for key in obj:
        if key.find("partition") ≠ -1:
            # vrne N seznamov, ki predstavljajo podatke o domenah
            dom_list = process_partition(obj[key])
            for items in dom_list:
                part_reports.append([pname, len(aidx), pid, *items])
            pid += 1

    fp.close()
    return part_reports

if __name__ == "__main__":
    with open(csv_name, "w", newline="") as csvfile:
        writer = csv.writer(csvfile)
        writer.writerow(field_names)
        for fname in glob.glob("sword_output/*.json"):
            report = process_report(fname)
            for line in report:
                writer.writerow(line)

# glej decompositions.r ...

```

2 ali več domen

```

library(dplyr)
library(ggplot2)

getwd()
# beri build_decompositions_csv.py za format
csv ← read.csv("sword_results.csv", header = TRUE) ▷
    as_tibble() ▷
    group_by(protein)
csv

# odstrani tiste, ki imajo samo eno domeno v optimalni particiji
multidomain ← csv ▷ filter(max(domain[partition == 0]) > 1)
multidomain

```

```

# A tibble: 12,386 × 8
# Groups:   protein [560]
  protein aindex partition quality domain  AUL start  end
  <chr>    <int>    <int>    <int> <int> <int> <int> <int>
1 1af7_A      3        0        3      1    56     1    80

```

```

2 1af7_A      3      0      3      2      86      81      274
3 1af7_A      3      1      1      1      58      1       80
4 1af7_A      3      1      1      2      60      81      155
5 1af7_A      3      1      1      3      88     156     194
6 1af7_A      3      1      1      4      66     195     274
7 1af7_A      3      2      1      1      60      1       80
8 1af7_A      3      2      1      2      46      81     243
9 1af7_A      3      2      1      3      88     156     194
10 1af7_A     3      2      1      4      66     244     274
# i 12,376 more rows
# i Use `print(n = ...)` to see more rows

```

```

# za začetek samo po optimalnih particijah
md_opt <- multidomain[multidomain$partition == 0, ]
proteins <- unique(md_opt$protein)

prot_group <- function(d, p)
{
  d[d$protein == p, ]
}

sexy_hist <- function(data, labl=TRUE, b=nclass.Sturges(data), xl="", yl="")
{
  # set layout
  lay_mat <- matrix(c(1, 2), nrow = 2)
  lay <- layout(mat = lay_mat, heights = c(1, 5))

  # plot
  par(mar = c(0, 3, 0, 0))
  boxplot(data,
    horizontal = TRUE,
    # ylim = c(min(data)-10, max(data)+10),
    axes = FALSE)

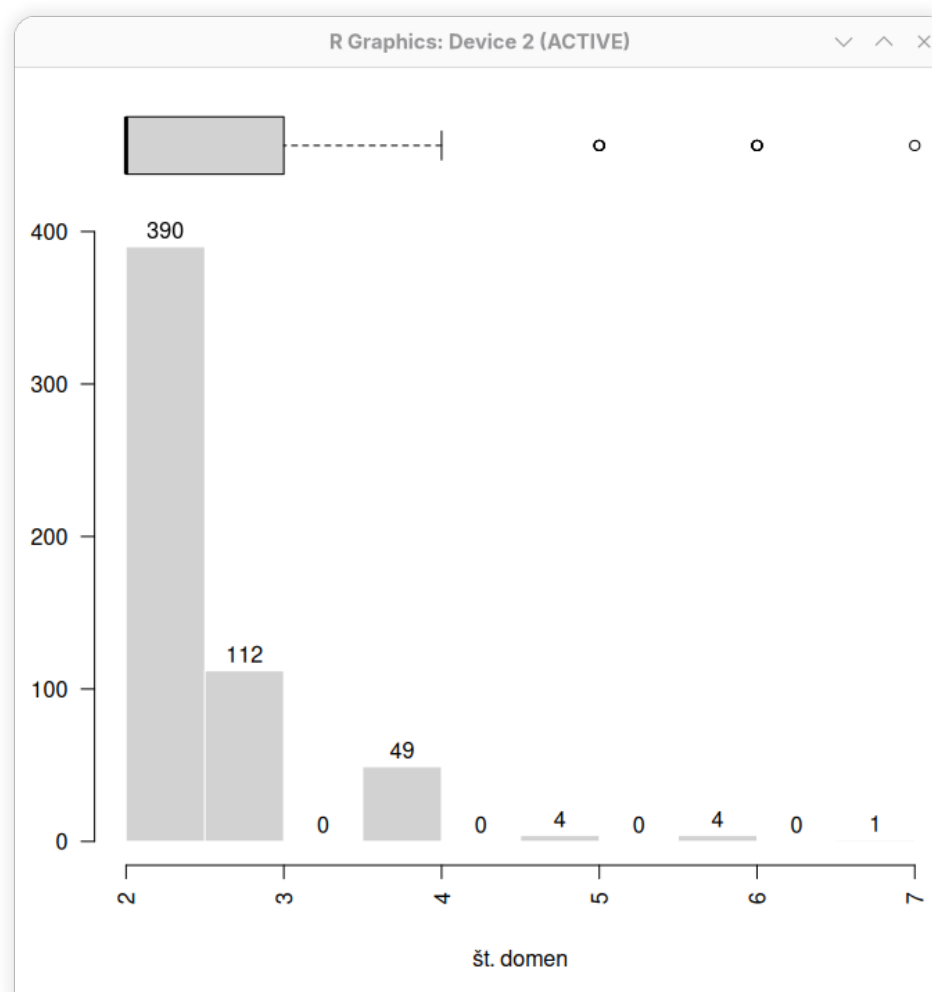
  par(mar = c(5, 3, 0, 0))
  hist(data,
    breaks = b,
    border = "white",
    # xlim = c(min(data)-10, max(data)+10),
    xlab = xl,
    ylab = yl,
    labels = labl,
    las = 2,
    main = "")
}

```

```

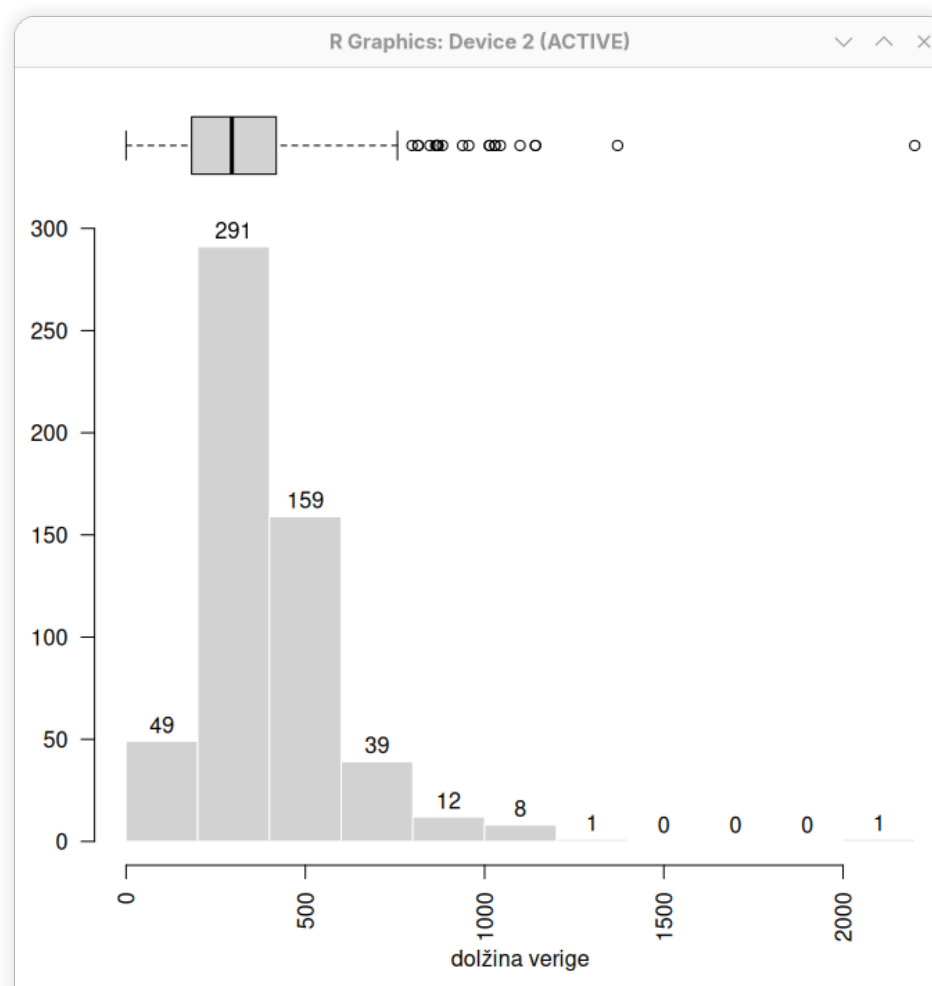
# TODO: kakšna je distribucija števila domen
md_opt$protein > table() > sexy_hist(xl = "št. domen")

```



```
# največ jih ima 2 domeni, kar je super za nas i think
which(table(md_opt$protein) == 2) > length()
# [1] 390
```

```
# TODO: kakšna je distribucija velikost proteinov po dolžini aminokislinske verige
ds <- c()
for (prot in proteins) {
  df <- prot_group(md_opt, prot)
  ds <- c(ds, max(df$end))
}
sexy_hist(ds, xl = "dolžina verige")
```



Verjetno imajo večji več domen.

razmerje približno 1:2 (max 1:3)

```
# ugotovi kateri kršijo razmerje 1:2, 1:3 v velikosti domen
# večje kot 1:3 je slabo
```

```
# iz start in end izračunaj velikost domene, primerjaj
check_ratios <- function(df, cutoff=0.67) {
  keep <- c()
  for (prot in proteins) {
    df <- prot_group(md_opt, prot) > as.data.frame() # tu nočem tibble

    # velikosti domen
    dsizes <- df$end - df$start + 1 # ker je 1-based

    # preveri ali katere kombinacije NE ustrezajo pogoju
    # kombinacije vseh domen: (1,2), (1,3), (2,3) ...
    # transponirano, da je dimenzije N×2
    m <- combn(df$domain, m = 2) > t()

    # kombinacija po kombinaciji, razmerje med domenama
    # razmerje manjša/večja domena, da je v intervalu [0,1]
    ratios <- apply(m, 1, \(x) min(dsizes[x]) / max(dsizes[x]) )

    # ohrani protein, če razmerja ustrezajo
    if (all(ratios < cutoff)) {
      keep <- c(keep, prot)
    }
  }
  keep
}
```

```
# manj strogo
keep <- check_ratios(md_opt)
length(keep)
length(proteins) - length(keep)
md_opt[which(md_opt$protein %in% keep), ]
write.csv(file = "ratio_1v3.csv",
  x = md_opt[which(md_opt$protein %in% keep), ],
  quote = FALSE,
  sep = ",",
  row.names = FALSE)
```

```
# A tibble: 662 × 8
# Groups:   protein [311]
  protein aindex partition quality domain AUL start end
  <chr>    <int>    <int>    <int>    <int> <int> <int> <int>
1 1af7_A      3        0        3        1    56     1    80
2 1af7_A      3        0        3        2    86    81   274
3 1c96_A      3        0        4        1    93     1   529
4 1c96_A      3        0        4        2    58   530   753
5 1d3y_A      3        0        4        1     3     1    74
6 1d3y_A      3        0        4        2    85    75   301
7 1d3y_B      3        0        4        1     0     1    74
8 1d3y_B      3        0        4        2    86    75   301
9 1dd3_A      1        0        4        1    83     1    49
10 1dd3_A      1        0        4        2    92    50   128
# i 652 more rows
# i Use `print(n = ...)` to see more rows
```

Ostalo je le še 311 proteinov ... Keep in mind, da so to tisti, katerih razmerje je največ 1:3. Če dam cutoff na 0.5, dobim

```
# bolj strogo
keep <- check_ratios(md_opt, cutoff = 0.5)
length(keep)
length(proteins) - length(keep)
write.csv(file = "ratio_1v2.csv",
  x = md_opt[which(md_opt$protein %in% keep), ],
  quote = FALSE,
  sep = ",",
  row.names = FALSE)
```

```
# A tibble: 367 × 8
# Groups:   protein [181]
  protein aindex partition quality domain AUL start end
  <chr>    <int>    <int>    <int>    <int> <int> <int> <int>
1 1af7_A      3        0        3        1    56     1    80
```

```

 2 1af7_A      3      0      3      2      86      81      274
 3 1c96_A      3      0      4      1      93       1      529
 4 1c96_A      3      0      4      2      58     530      753
 5 1d3y_A      3      0      4      1       3       1       74
 6 1d3y_A      3      0      4      2     85      75     301
 7 1d3y_B      3      0      4      1       0       1       74
 8 1d3y_B      3      0      4      2     86      75     301
 9 1dj0_B      2      0      4      1     89       1     264
10 1dj0_B      2      0      4      2     75     113     239
# i 357 more rows
# i Use `print(n = ...)` to see more rows

```

181 :D ... Also keep in mind, da imaš trenutno samo optimalne particije. Morda, res morda, se v nekaterih primerih splača vzeti neko, ki ni optimalna. Ampak nevem še kdaj točno.

```

md_keep <- read.csv(file = "ratio_1v2.csv", sep = ",", header = TRUE) >
  as_tibble() >
  group_by(protein)

```

dober A-index

```

# do 3 je okej in my book
md_keep$aindex > table()

```

```

 1    2    3
14 134 219

```

dobre AUL vrednosti

```

# če je slaba potem odstrani... ampak slaba je kdaj? manj kot 50..?
# odvisno od podatkov..? Ali se sploh splača naprej? Preveri porazdelitev

# NOTE: vzame vse AUL vrednosti, moral bi vzeti min/max/mean vsakega proteina (?)
# moram se znebit vseh proteinov, ki imajo v katerikoli domeni AUL manjši od cutoff
# gledam porazdelitev "najslabših" domen med proteini, vsak protein ima enega predstavnika

bad_doms <- data.frame()

for (prot in unique(md_keep$protein)) {
  df <- prot_group(md_keep, prot)

  # najdi najslabšo domeno
  bad_dom <- which(df$AUL == min(df$AUL))[1]

  # dodaj na seznam
  bad_doms <- rbind(bad_doms, df[bad_dom, ])
}

# vsak protein je notri
all(group_keys(bad_doms) == group_keys(md_keep))

# ni duplikatov
all(bad_doms$protein == unique(bad_doms$protein))

bad_doms

```

```

# A tibble: 181 × 8
# Groups:   protein [181]
  protein aindex partition quality domain  AUL start  end
  <chr>    <int>    <int>    <int> <int> <int> <int> <int>
1 1af7_A      3      0      3      1     56     1    80
2 1c96_A      3      0      4      2     58    530   753
3 1d3y_A      3      0      4      1      3     1    74
4 1d3y_B      3      0      4      1      0     1    74
5 1dj0_B      2      0      4      2     75    113   239
6 1ewf_A      3      0      4      2     68    267   427
7 1fqi_A      2      0      3      1     18     1   147
8 1gpj_A      3      0      3      3     84    330   404
9 1hbn_E      3      0      4      1     63     1   442

```

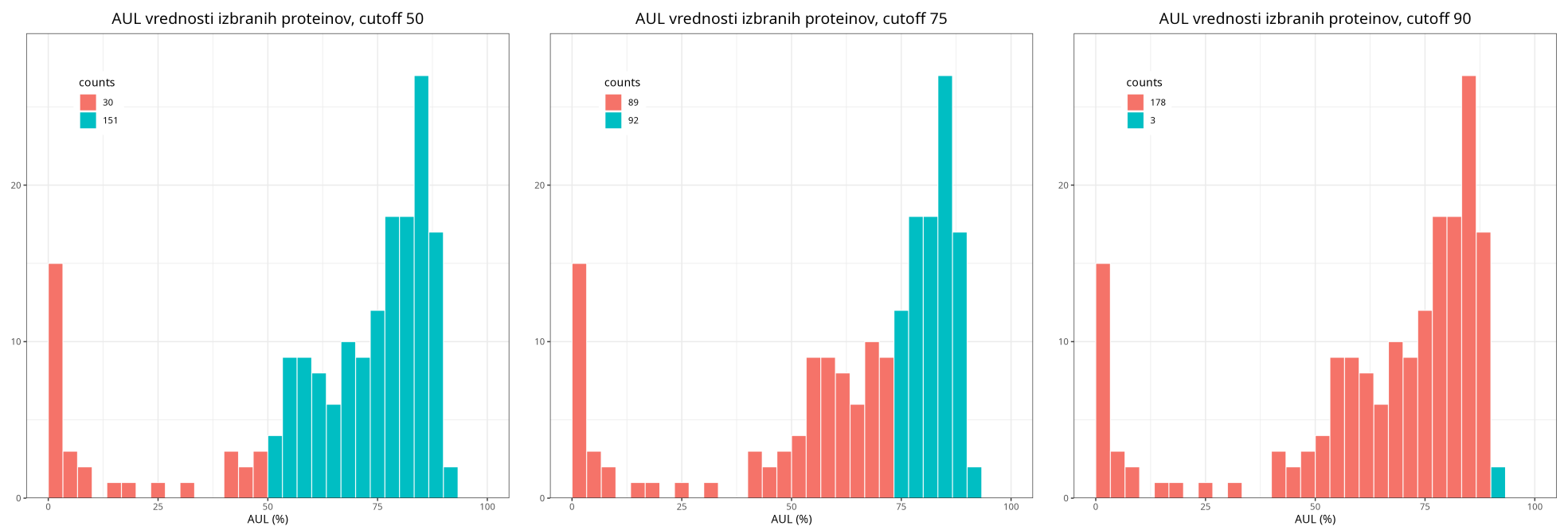
```
10 1hw1_B      3      0      4      1    77      1    78
# i 171 more rows
# i Use `print(n = ...)` to see more rows
```

```
for (c in c(50, 75, 90))
{
  cutoff ← c

  n_below ← sum(bad_doms$AUL < cutoff)
  n_above ← sum(bad_doms$AUL ≥ cutoff)

  ggplot(bad_doms, aes(x = AUL)) +
    geom_histogram(aes(fill = after_stat(x) ≥ cutoff),
      breaks = seq(0, 100, length.out = 31),
      color = "white",
      linewidth = 0.4) +
    # stat_bin(aes(label = after_stat(count)),
    #         geom = "text",
    #         breaks = seq(0, 100, length.out = 31),
    #         vjust = -0.5,
    #         size = 3.5) +
    scale_fill_hue(labels = c(n_below, n_above)) +
    scale_y_continuous(expand = expansion(mult = c(0, 0.1))) +
    labs(title = paste("AUL vrednosti izbranih proteinov, cutoff", cutoff),
      x = "AUL (%)",
      y = "",
      fill = "counts") +
    theme_bw() +
    theme(plot.title = element_text(hjust = 0.5, size = 15),
      legend.position = c(0.15, 0.85),
      legend.background = element_rect(fill = "white"),
    )
  ggsave(filename = paste0("aul", cutoff, ".png"))
}
```

```
$ magick aul50.png aul75.png aul90.png +append aul_distribution.png
```



Note da niso najboljši rezultati. Samo 3 proteini ostanejo z odličnimi domenami. 92 jih ostane z dobrimi, 151 z zadostnimi.

Kočni dataset

```
# TODO: ohrani vse ki so dobri
selected ← bad_doms[which(bad_doms$AUL ≥ 75), ]$protein
final ← filter(md_keep, protein %in% selected)

write.csv(file = "clean_dataset.csv",
  x = final,
  sep = ",",
  quote = FALSE,
  row.names = FALSE)
```